

在 Cloud Run 部署 Django CMS

來源：<https://codelabs.developers.google.com/codelabs/cloud-run-djangocms?hl=zh-tw>

步驟數：11

在本程式碼研究室中，您將瞭解如何使用無伺服器元件部署 Django CMS：網路引擎適用的 Cloud Run、資料庫適用的 Cloud SQL，以及媒體資產適用的 Cloud Build。

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 啟用 Cloud API](#)
4. [4. 建立範本專案](#)
5. [5. 建立後端服務](#)
6. [6. 設定應用程式](#)
7. [7. 設定、建構及執行遷移步驟](#)
8. [8. 部署至 Cloud Run](#)
9. [9. 存取 Django 管理員](#)
10. [10. 套用應用程式更新](#)
11. [11. 恭喜！](#)

1. 簡介



[Cloud Run](#) 是代管運算平台，能讓您執行可透過 HTTP 要求叫用的無狀態容器。Cloud Run 採用無伺服器技術，為您省去所有基礎架構管理工作，讓您專心處理最重要的事：建構出色應用程式。

此外，它還能與 Google Cloud 生態系統的許多其他部分進行原生介面互動，包括用於代管資料庫的 [Cloud SQL](#)、用於統一物件儲存空間的 [Cloud Storage](#)，以及用於管理密鑰的 [Secret Manager](#)。

[Django CMS](#) 是以 Django 為基礎建構的企業內容管理系統 (CMS)。[Django](#) 是高階 Python 網頁架構。

在本教學課程中，您將使用這些元件部署小型 Django CMS 專案。

注意：本程式碼研究室最後一次驗證時，使用的是 [Django CMS 4.1.2](#) 和 [django-cms/cms-template v4.1](#)。

課程內容

- 如何使用 Cloud Shell
 - 如何建立 Cloud SQL 資料庫
 - 如何建立 Cloud Storage bucket
 - 如何建立 Secret Manager 密鑰
 - 如何使用不同 Google Cloud 服務的密鑰
 - 如何將 Google Cloud 元件連線至 Cloud Run 服務
 - 如何使用 Container Registry 儲存建構的容器
 - 如何部署至 Cloud Run
 - 如何在 Cloud Build 中執行資料庫結構定義遷移作業
-

步驟 2 · 約 5 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The image shows two screenshots from the Google Cloud console. The top screenshot shows the 'Select a project' dropdown menu in the top navigation bar, with a blue arrow pointing to it. The bottom screenshot shows the 'New Project' form. The 'Project name' field contains 'My Project 13475'. The 'Project ID' field contains 'inbound-analogy-389614', which is highlighted with a blue box. The 'Location' field is empty, and the 'BROWSE' button is visible. The 'CREATE' and 'CANCEL' buttons are at the bottom.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使

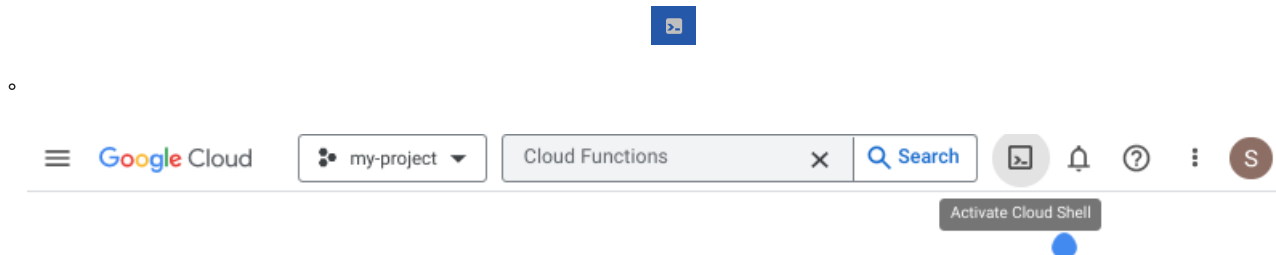
用者可參加[價值\\$300 美元的免費試用計畫](#)。

Google Cloud Shell

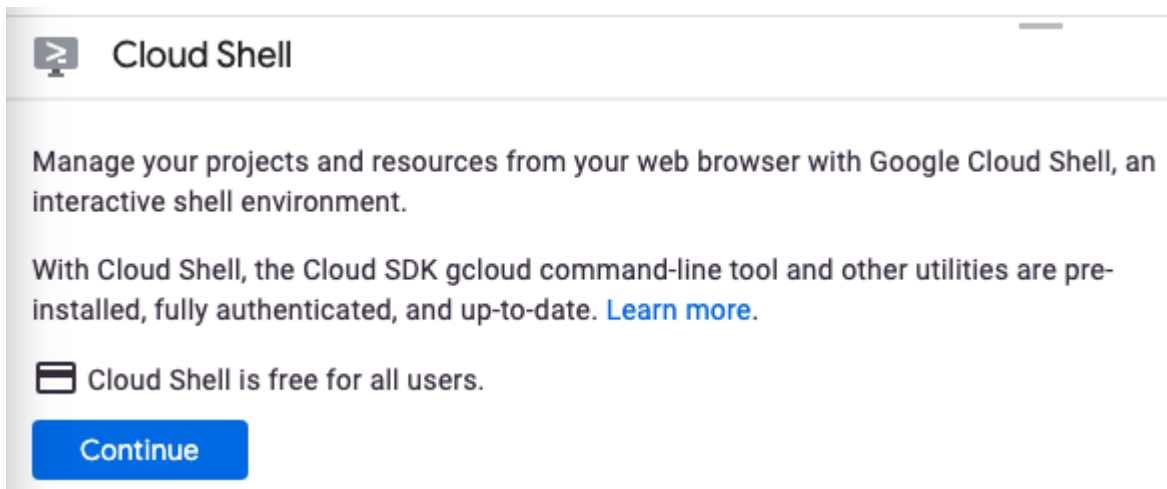
雖然您可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，我們將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

啟用 Cloud Shell

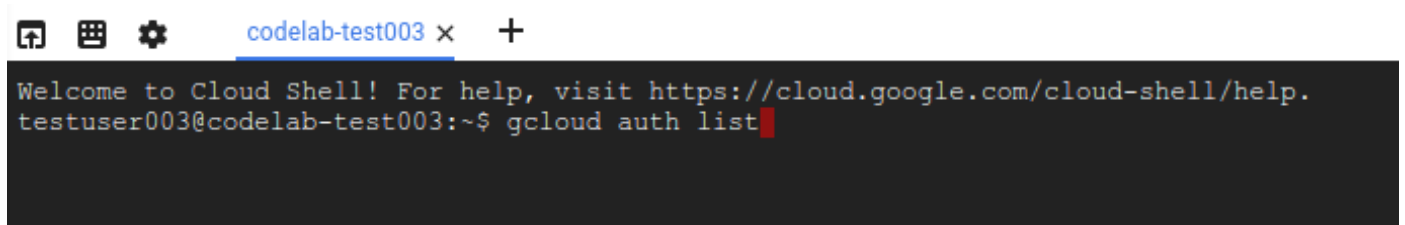
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面，說明這個指令列環境。如果出現中繼畫面，請按一下「繼續」。



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器已載入所有必要的開發工具，並提供永久的 5 GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

Credentialed Accounts

ACTIVE ACCOUNT

* <my_account>@<my_domain.com>

To set the active account, run:

```
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。第一次執行指令時，系統可能會提示您進行驗證。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

注意： 如果您在 Cloud Shell 以外的環境完成本教學課程，請按照「[設定應用程式預設憑證](#)」一文的說明操作。

步驟 3 · 約 0 分鐘

3. 啟用 Cloud API

在 Cloud Shell 中，為要使用的元件啟用 Cloud API：

```
gcloud services enable \  
  run.googleapis.com \  
  sql-component.googleapis.com \  
  sqladmin.googleapis.com \  
  compute.googleapis.com \  
  cloudbuild.googleapis.com \  
  secretmanager.googleapis.com \  
  artifactregistry.googleapis.com
```

由於這是您第一次從 gcloud 呼叫 API，系統會要求您授權使用憑證發出這項要求。每個 Cloud Shell 工作階段只會發生一次。

這項作業可能需要一些時間才能完成。

完成後，畫面應會顯示類似以下的成功訊息：

```
Operation "operations/acf.cc11852d-40af-47ad-9d59-477a12847c9e" finished  
successfully.
```

步驟 4 · 約 3 分鐘

4. 建立範本專案

您將使用 Django CMS [cms-template](#) 做為 Django CMS 專案範例。

如要建立這個範本專案，請使用 Cloud Shell 建立名為 `djangocms-cloudrun` 的新目錄，然後前往該目錄：

```
mkdir ~/djangocms-cloudrun
cd ~/djangocms-cloudrun
```

將 django-cms 套件安裝到臨時虛擬環境中：

```
virtualenv venv
source venv/bin/activate
pip install djangocms-frontend\[cms-4]
```

複製 cms-template 專案：

```
django-admin startproject --template https://github.com/django-cms/cms-
template/archive/4.1.zip myproject .
```

將 `requirements.in` 檔案重新命名為 `requirements.txt`。(`.in` 檔案是由 [pip-tools](#) 產生 `requirements.txt` 檔案，但如果變更副檔名，即可直接使用。在後續步驟中，`pip` 會預期 `.txt` 擴充功能。)

```
mv requirements.in requirements.txt
```

現在您會在名為 `myproject` 的資料夾中，看到 Django CMS 專案範本：

```
ls -F
```

```
manage.py*  media/  myproject/  project.db requirements.txt  static/  venv/
```

現在可以退出並移除臨時虛擬環境：

```
deactivate
rm -rf venv
```

從這裡，系統會在容器內呼叫 Django CMS。

步驟 5 · 約 5 分鐘

5. 建立後端服務

現在要建立後端服務：專屬服務帳戶、Artifact Registry、Cloud SQL 資料庫、Cloud Storage bucket，以及多個 Secret Manager 值。

確保部署作業中使用的密碼值安全無虞，對任何專案的安全性都至關重要，可避免使用者不慎將密碼放在不該放置的位置 (例如直接放在設定檔中，或直接輸入終端機，導致密碼可從記錄中擷取)。

首先，請設定兩個基本環境變數，一個用於專案 ID：

```
PROJECT_ID=$(gcloud config get-value core/project)
```

以及區域：

```
REGION=us-central1
```

處理基礎架構時，最好將所有元件放在同一位置，這樣元件之間就能更有效率地通訊。部分元件適用於全球，但並非所有元件都適用於所有地區。

本程式碼研究室建議使用 `us-central1`、`eu-central-1` 或 `asia-northeast1`，或是 Cloud Run (全代管) 支援的任何區域。

瞭解詳情：

- [Cloud Run \(全代管\) 位置](#)
- [Cloud SQL 位置](#)
- [Cloud Storage 位置](#)

建立服務帳戶

如要限制服務對 Google Cloud 其他部分的存取權，請建立專屬服務帳戶：

```
gcloud iam service-accounts create cloudrun-serviceaccount
```

在本程式碼研究室的後續章節中，您會透過電子郵件參照這個帳戶。在環境變數中設定該值：

```
SERVICE_ACCOUNT=$(gcloud iam service-accounts list \
  --filter cloudrun-serviceaccount --format "value(email)")
```

建立 Artifact Registry

如要儲存建構的容器映像檔，請在所選區域建立容器登錄：

```
gcloud artifacts repositories create containers --repository-format docker --location $REGION
```

您會在後續的程式碼研究室中，依名稱參照這個登錄檔：

```
ARTIFACT_REGISTRY=${REGION}-docker.pkg.dev/${PROJECT_ID}/containers
```

建立資料庫

建立 Cloud SQL 執行個體：

```
gcloud sql instances create myinstance --project $PROJECT_ID \
  --database-version POSTGRES_14 --tier db-f1-micro --region $REGION
```

這項作業可能需要幾分鐘才能完成。

在該執行個體中建立資料庫：

```
gcloud sql databases create mydatabase --instance myinstance
```

在同一個執行個體中建立使用者：

```
DJPASS="$(cat /dev/urandom | LC_ALL=C tr -dc 'a-zA-Z0-9' | fold -w 30 | head -n 1)"
gcloud sql users create djuser --instance myinstance --password $DJPASS
```

授予服務帳戶連線至執行個體的權限：

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member serviceAccount:${SERVICE_ACCOUNT} \
  --role roles/cloudsql.client
```

建立儲存空間 bucket

建立 Cloud Storage bucket (請注意，名稱必須是全域不重複)：

```
GS_BUCKET_NAME=${PROJECT_ID}-media
gcloud storage buckets create gs://${GS_BUCKET_NAME} --location ${REGION}
```

授予服務帳戶管理 bucket 的權限：

```
gcloud storage buckets add-iam-policy-binding gs://${GS_BUCKET_NAME} \
  --member serviceAccount:${SERVICE_ACCOUNT} \
  --role roles/storage.admin
```

由於儲存在值區中的物件來源不同 (值區網址而非 Cloud Run 網址)，您需要設定[跨源資源共享 \(CORS\)](#) 設定。

建立名為 `cors.json` 的新檔案，並在其中加入下列內容：

```
touch cors.json
cloudshell edit cors.json
```

cors.json

```
[
  {
    "origin": ["*"],
    "responseHeader": ["Content-Type"],
    "method": ["GET"],
    "maxAgeSeconds": 3600
  }
]
```

將這項 CORS 設定套用至新建立的儲存空間值區：

```
gsutil cors set cors.json gs://$GS_BUCKET_NAME
```

將設定儲存為密鑰

設定支援服務後，您現在要將這些值儲存在受 Secret Manager 保護的檔案中。

Secret Manager 可讓您以二進位 blob 或文字字串的形式儲存、管理及存取密鑰。適用於儲存設定資訊，例如資料庫密碼、API 金鑰或應用程式在執行階段所需的 TLS 憑證。

首先，請建立檔案，並填入[資料庫連線字串](#)、媒體儲存空間、Django 的密鑰 (用於加密簽署工作階段和權杖) 的值，並啟用偵錯功能：

```
echo
DATABASE_URL="\postgres://djuser:${DJPASS}@//cloudsql/${PROJECT_ID}:${REGION}:myinstance/mydatabase\" > .env

echo GS_BUCKET_NAME="\${GS_BUCKET_NAME}\" >> .env

echo SECRET_KEY="\$(cat /dev/urandom | LC_ALL=C tr -dc 'a-zA-Z0-9' | fold -w 50 |
head -n 1)\\" >> .env

echo DEBUG=True >> .env
```

接著，使用該檔案做為密鑰，建立名為 `application_settings` 的密鑰：

```
gcloud secrets create application_settings --data-file .env
```

允許服務帳戶存取這個密鑰：

```
gcloud secrets add-iam-policy-binding application_settings \  
  --member serviceAccount:${SERVICE_ACCOUNT} --role  
roles/secretmanager.secretAccessor
```

列出密鑰，確認密鑰已建立：

```
gcloud secrets versions list application_settings
```

確認已建立密鑰後，請移除本機檔案：

```
rm .env
```

步驟 6 · 約 3 分鐘

6. 設定應用程式

根據您剛建立的支援服務，您需要對範本專案進行一些變更，以符合需求。

包括介紹如何使用 `django-environ` 將環境變數做為設定，並以您定義為密鑰的值做為種子。如要實作這項功能，請擴充範本設定。您也需要新增額外的 Python 依附元件。

設定

移動 `settings.py` 檔案，並重新命名為 `basesettings.py`：

```
mv myproject/settings.py myproject/basesettings.py
```

使用 Cloud Shell 網頁編輯器建立新的 `settings.py` 檔案，並加入下列程式碼：

```
touch myproject/settings.py
cloudshell edit myproject/settings.py
```

myproject/settings.py

```

import io
import os
from urllib.parse import urlparse

import environ

# Import the original settings from each template
from .basesettings import *

# Load the settings from the environment variable
env = environ.Env()
env.read_env(io.StringIO(os.environ.get("APPLICATION_SETTINGS", None)))

# Setting this value from django-environ
SECRET_KEY = env("SECRET_KEY")

# Ensure myproject is added to the installed applications
if "myproject" not in INSTALLED_APPS:
    INSTALLED_APPS.append("myproject")

# If defined, add service URLs to Django security settings
CLOUDRUN_SERVICE_URLS = env("CLOUDRUN_SERVICE_URLS", default=None)
if CLOUDRUN_SERVICE_URLS:
    CSRF_TRUSTED_ORIGINS = env("CLOUDRUN_SERVICE_URLS").split(",")
    # Remove the scheme from URLs for ALLOWED_HOSTS
    ALLOWED_HOSTS = [urlparse(url).netloc for url in CSRF_TRUSTED_ORIGINS]
else:
    ALLOWED_HOSTS = ["*"]

# Default false. True allows default landing pages to be visible
DEBUG = env("DEBUG", default=False)

# Set this value from django-environ
DATABASES = {"default": env.db()}

# Change database settings if using the Cloud SQL Auth Proxy
if os.getenv("USE_CLOUD_SQL_AUTH_PROXY", None):
    DATABASES["default"]["HOST"] = "127.0.0.1"
    DATABASES["default"]["PORT"] = 5432

# Define static storage via django-storages[google]
GS_BUCKET_NAME = env("GS_BUCKET_NAME")
STATICFILES_DIRS = []
GS_DEFAULT_ACL = "publicRead"
STORAGES = {
    "default": {
        "BACKEND": "storages.backends.gcloud.GoogleCloudStorage",
    },
    "staticfiles": {
        "BACKEND": "storages.backends.gcloud.GoogleCloudStorage",
    },
}

```

```
},  
}
```

請花時間閱讀針對各項設定新增的註解。

請注意，您可能會在這個檔案中看到 Lint 錯誤。這個狀況有可能發生：Cloud Shell 沒有這個專案的需求條件內容，因此可能會回報無效的匯入項目和未使用的匯入項目。

Python 依附元件

找出 `requirements.txt` 檔案，並附加下列套件：

```
cloudshell edit requirements.txt
```

requirements.txt (附加)

```
gunicorn  
psycpg2-binary  
django-storages[google]  
django-envron
```

定義應用程式映像檔

只要容器符合 [Cloud Run 容器合約](#)，Cloud Run 就會執行。本教學課程選擇省略 `Dockerfile`，改用 [Cloud Native Buildpacks](#)。建構包可協助建構常見語言 (包括 Python) 的容器。

本教學課程選擇自訂用於啟動網頁應用程式的 `Procfile`。

如要將範本專案容器化，請先在專案頂層 (與 `manage.py` 位於相同目錄) 建立名為 `Procfile` 的新檔案，然後複製下列內容：

```
touch Procfile  
cloudshell edit Procfile
```

Procfile

```
web: gunicorn --bind 0.0.0.0:$PORT --workers 1 --threads 8 --timeout 0  
myproject.wsgi:application
```

步驟 7 · 約 3 分鐘

7. 設定、建構及執行遷移步驟

如要在 Cloud SQL 資料庫中建立資料庫結構定義，並在 Cloud Storage bucket 中填入靜態資產，您需要執行 `migrate` 和 `collectstatic`。

這些 Django 遷移基本指令必須在已建構的容器映像檔環境中執行，且必須有資料庫存取權。

您也需要執行 `createsuperuser`，建立管理員帳戶來登入 Django 管理員。

為此，您將使用 Cloud Run Jobs 執行這些工作。Cloud Run 工作可讓您執行有明確結尾的程序，因此非常適合用於管理工作。

定義 Django 超級使用者密碼

如要建立超級使用者，請使用 `createsuperuser` 指令的非互動式版本。這項指令需要使用特別命名的環境變數，取代輸入密碼的提示。

使用隨機產生的密碼建立新密鑰：

```
echo -n $(cat /dev/urandom | LC_ALL=C tr -dc 'a-zA-Z0-9' | fold -w 30 | head -n 1) |  
gcloud secrets create django_superuser_password --data-file=-
```

允許服務帳戶存取這個密鑰：

```
gcloud secrets add-iam-policy-binding django_superuser_password \  
  --member serviceAccount:${SERVICE_ACCOUNT} \  
  --role roles/secretmanager.secretAccessor
```

更新 Procfile

為清楚瞭解 Cloud Run 作業，請在 Procfile 中建立捷徑，並將下列進入點附加至 `Procfile`：

```
migrate: python manage.py migrate && python manage.py collectstatic --noinput --clear  
createuser: python manage.py createsuperuser --username admin --email  
noop@example.com --noinput
```

您現在應該有三個項目：預設的 `web` 進入點、套用資料庫遷移的 `migrate` 進入點，以及執行 `createsuperuser` 指令的 `createuser` 進入點。

建構應用程式映像檔

更新 Procfile 後，請建構映像檔：

```
gcloud builds submit --pack image=${ARTIFACT_REGISTRY}/myimage
```

建立 Cloud Run 工作

映像檔建立完成後，您就可以使用該映像檔建立 Cloud Run 工作。

這些工作會使用先前建構的映像檔，但會使用不同的 `command` 值。這些會對應至 `Procfile` 中的值。

建立遷移作業：

```
gcloud run jobs create migrate \  
  --region $REGION \  
  --image ${ARTIFACT_REGISTRY}/myimage \  
  --set-cloudsql-instances ${PROJECT_ID}:${REGION}:myinstance \  
  --set-secrets APPLICATION_SETTINGS=application_settings:latest \  
  --service-account $SERVICE_ACCOUNT \  
  --command migrate
```

為使用者建立作業：

```
gcloud run jobs create createuser \  
  --region $REGION \  
  --image ${ARTIFACT_REGISTRY}/myimage \  
  --set-cloudsql-instances ${PROJECT_ID}:${REGION}:myinstance \  
  --set-secrets APPLICATION_SETTINGS=application_settings:latest \  
  --set-secrets DJANGO_SUPERUSER_PASSWORD=django_superuser_password:latest \  
  --service-account $SERVICE_ACCOUNT \  
  --command createuser
```

執行 Cloud Run 工作

設定好工作後，請執行遷移作業：

```
gcloud run jobs execute migrate --region $REGION --wait
```

確認這項指令的輸出內容顯示執行作業「成功完成」。

您稍後更新應用程式時，會執行這個指令。

設定資料庫後，請使用作業建立使用者：

```
gcloud run jobs execute createuser --region $REGION --wait
```

確認這項指令的輸出內容顯示執行作業「成功完成」。

您不必再次執行這項指令。

步驟 8 · 約 2 分鐘

8. 部署至 Cloud Run

建立並填入支援服務後，您現在可以建立 Cloud Run 服務來存取這些服務。

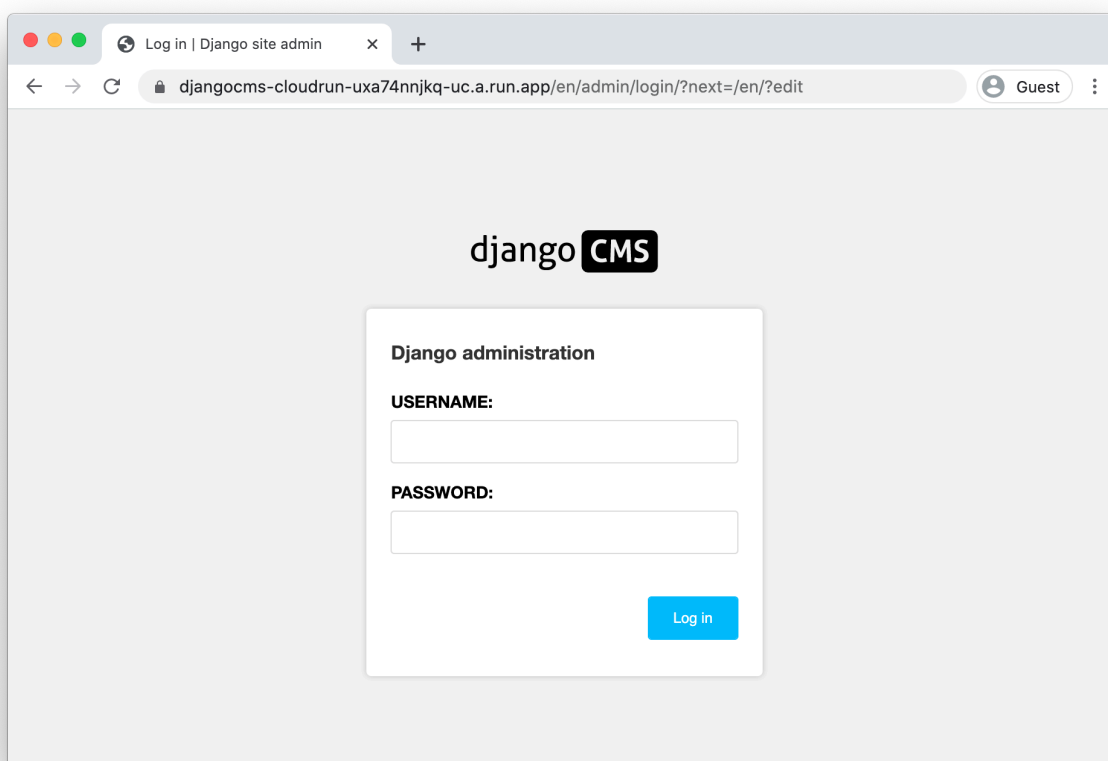
使用下列指令，將容器化應用程式初步部署至 Cloud Run：

```
gcloud run deploy.djangocms-cloudrun \
  --region $REGION \
  --image ${ARTIFACT_REGISTRY}/myimage \
  --set-cloudsql-instances ${PROJECT_ID}:${REGION}:myinstance \
  --set-secrets APPLICATION_SETTINGS=application_settings:latest \
  --service-account $SERVICE_ACCOUNT \
  --allow-unauthenticated
```

稍候片刻，等待部署作業完成。部署成功之後，指令列會顯示服務網址：

```
Service [djangocms-cloudrun] revision [djangocms-cloudrun-00001-...] has been
deployed and is serving 100 percent of traffic.
Service URL: https://djangocms-cloudrun-...run.app
```

您現在可以在網路瀏覽器中開啟這個網址，造訪您所部署的容器：



由於這是新安裝的應用程式，系統會自動將您重新導向至登入頁面。

9. 存取 Django 管理員

Django CMS 的主要功能之一是互動式管理介面。

更新 CSRF 設定

Django 4.0 的變更：現在系統會強制執行 CSRF Origin 標頭檢查。請參閱 [Django 說明文件](#)。

Django 內建防範跨網站偽造要求 (CSRF) 的機制。每當 Django 網站提交表單 (包括登入 Django 管理員)，系統都會檢查「信任來源」設定。如果不符，Django 會傳回錯誤。

在 `mysite/settings.py` 檔案中，如果定義了 `CLOUDRUN_SERVICE_URL` 環境變數，系統就會在 `CSRF_TRUSTED_ORIGINS` 和 `ALLOWED_HOSTS` 設定中使用該變數。雖然定義 `ALLOWED_HOSTS` 並非必要，但建議您新增此項目，因為 `CSRF_TRUSTED_ORIGINS` 已規定必須加入此項目。

由於您需要服務網址，因此必須先完成首次部署，才能新增這項設定。

您必須更新服務才能新增這項環境變數。可以新增至 `application_settings` Secret，或直接新增為環境變數。

下方的實作方式會利用 gcloud [格式](#)和[逸出](#)。

擷取服務網址：

```
CLOUDRUN_SERVICE_URLS=$(gcloud run services describe.djangocms-cloudrun \
  --region $REGION \
  --format "value(metadata.annotations[\"run.googleapis.com/urls\"])" | tr -d '"[]')
echo $CLOUDRUN_SERVICE_URLS
```

在 Cloud Run 服務中，將這個值設為環境變數：

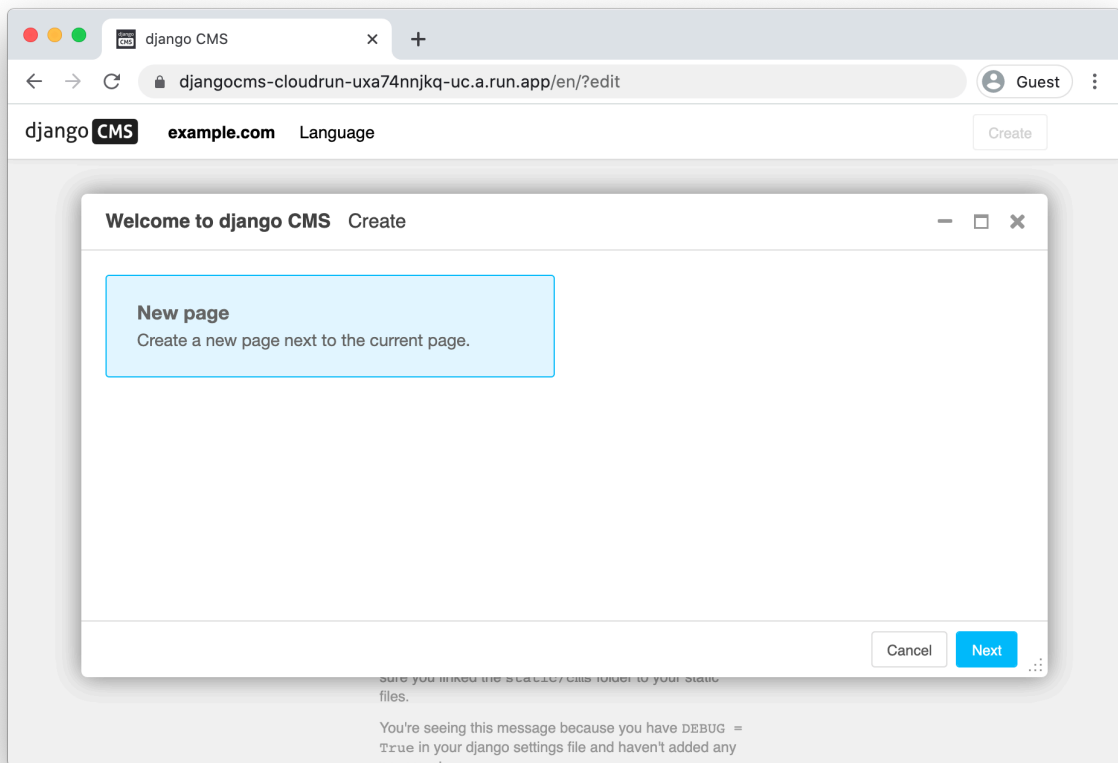
```
gcloud run services update.djangocms-cloudrun \
  --region $REGION \
  --update-env-vars "^##^CLOUDRUN_SERVICE_URLS=$CLOUDRUN_SERVICE_URLS"
```

登入 Django 管理員

若要存取 Django 管理介面，請在服務網址後方加上 `/admin`。

現在使用使用者名稱「admin」登入，並使用下列指令擷取密碼：

```
gcloud secrets versions access latest --secret django_superuser_password && echo ""
```



步驟 10 · 約 2 分鐘

10. 套用應用程式更新

開發應用程式時，您會想在本機測試。如要這麼做，您必須連線至 Cloud SQL (「正式環境」) 資料庫或本機 (「測試」) 資料庫。

連線至實際工作環境資料庫

您可以使用 [Cloud SQL 驗證 Proxy](#) 連線至 Cloud SQL 執行個體。這個應用程式會建立從本機電腦到資料庫的連線。

安裝 Cloud SQL 驗證 Proxy 後，請按照下列步驟操作：

```
# Create a virtualenv
virtualenv venv
source venv/bin/activate
pip install -r requirements.txt

# Copy the application settings to your local machine
gcloud secrets versions access latest --secret application_settings > temp_settings

# Run the Cloud SQL Auth Proxy
./cloud-sql-proxy ${PROJECT_ID}:${REGION}:myinstance

# In a new tab, start the local web server using these new settings
USE_CLOUD_SQL_AUTH_PROXY=true APPLICATION_SETTINGS=$(cat temp_settings) python
manage.py runserver
```

完成作業後，請務必移除 `temp_settings` 檔案。

連線至本機 SQLite 資料庫

或者，您也可以開發應用程式時使用本機資料庫。Django 支援 PostgreSQL 和 SQLite 資料庫，PostgreSQL 具備 SQLite 沒有的某些功能，但在許多情況下，兩者的功能相同。

如要設定 SQLite，您必須更新應用程式設定，指向本機資料庫，然後套用結構定義遷移作業。

如要設定這項付款方式，請按照下列步驟操作：

```
# Create a virtualenv
virtualenv venv
source venv/bin/activate
pip install -r requirements.txt

# Copy the application settings to your local machine
gcloud secrets versions access latest --secret application_settings > temp_settings

# Edit the DATABASE_URL setting to use a local sqlite file. For example:
DATABASE_URL=sqlite:///tmp/my-tmp-sqlite.db

# Set the updated settings as an environment variable
APPLICATION_SETTINGS=$(cat temp_settings)

# Apply migrations to the local database
python manage.py migrate

# Start the local web server
python manage.py runserver
```

完成作業後，請務必移除 `temp_settings` 檔案。

建立遷移作業

變更資料庫模型時，您可能需要執行 `python manage.py makemigrations`，產生 Django 的遷移檔案。

設定正式版或測試版資料庫連線後，即可執行這項指令。或者，您也可以提供空白設定，在沒有資料庫的情況下產生遷移檔案：

```
SECRET_KEY="" DATABASE_URL="" GS_BUCKET_NAME="" python manage.py makemigrations
```

套用應用程式更新

如要對應用程式套用變更，請按照下列步驟操作：

- 將變更內容建構為新圖片，
- 套用任何資料庫或靜態遷移作業，然後
- 更新 Cloud Run 服務，以便使用新映像檔。

如要建構映像檔，請按照下列步驟操作：

```
gcloud builds submit --pack image=${ARTIFACT_REGISTRY}/myimage
```

如有任何要套用的遷移作業，請執行 Cloud Run 工作：

```
gcloud run jobs execute migrate --region $REGION --wait
```

如要使用新映像檔更新服務，請按照下列步驟操作：

```
gcloud run services update.djangocms-cloudrun \  
  --platform managed \  
  --region $REGION \  
  --image gcr.io/${PROJECT_ID}/myimage
```

11. 恭喜！

您已將複雜專案部署到 Cloud Run！

- Cloud Run 會自動水平擴充容器映像檔以處理收到的要求，並在需求減少時縮減規模。您只需要支付處理要求期間使用的 CPU、記憶體和網路費用。
- Cloud SQL 可讓您佈建代管的 PostgreSQL 執行個體，系統會自動維護，並原生整合至許多 Google Cloud 系統。
- Cloud Storage 可讓您以 Django 順暢存取的方式使用雲端儲存空間。
- Secret Manager 可讓您儲存密鑰，並允許 Google Cloud 的特定部分存取密鑰，其他部分則無法存取。

雖然本實驗室是使用 `gcloud` 指令列完成，但您也可以透過 Cloud 控制台 (console.cloud.google.com) 使用 Cloud Run。只要依序前往 `Compute` > `Cloud Run`，即可查看服務清單。

清理

如要避免系統向您的 Google Cloud Platform 帳戶收取您在本教學課程中所用資源的相關費用：

- 前往 Cloud Console 中的「管理資源」頁面。
- 在專案清單中選取專案，然後按一下「刪除」。
- 在對話方塊中輸入專案 ID，然後按一下「Shut down」（關閉）刪除專案。

瞭解詳情

- 在 Cloud Run 中執行 Django：<https://cloud.google.com/python/django/run>
 - 使用 Python 執行 Hello Cloud Run：<https://codelabs.developers.google.com/codelabs/cloud-run-hello-python3>
 - Google Cloud 上的 Python：<https://cloud.google.com/python>
 - Google Cloud Python 用戶端：<https://github.com/googleapis/google-cloud-python>
-